Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

GEBZE TEKNİK ÜNİVERSİTESİ

Lojik Devreler ve Tasarım

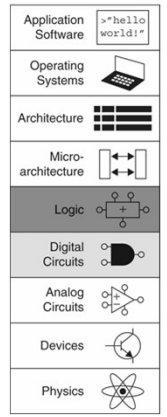
Bahar 2023
Dr. Öğr. Ü. İhsan Çiçek

4. Boole Cebri ve Teoremleri, Lojik Sadeleştirme

1

Chapter 2 :: Topics

- **Introduction**
- **Boolean Equations**
- **Boolean Algebra**
- **From Logic to Gates**
- **Multilevel Combinational Logic**
- **X's and Z's, Oh My**
- **Karnaugh Maps**
- **Combinational Building Blocks**
- **Timing**

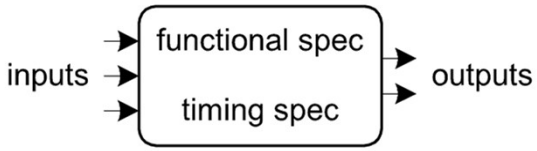


2

Introduction

A logic circuit is composed of:

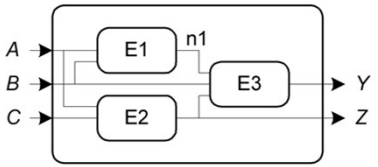
- Inputs
- Outputs
- Functional specification
- Timing specification



3

Circuits

- Nodes
  - Inputs: *A*, *B*, *C*
  - Outputs: *Y*, *Z*
  - Internal: n1
- Circuit elements
  - E1, E2, E3
  - Each a circuit

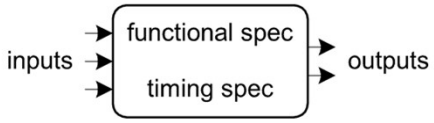


4

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Types of Logic Circuits

- **Combinational Logic**
  - Memoryless
  - Outputs determined by current values of inputs
- **Sequential Logic**
  - Has memory
  - Outputs determined by previous and current values of inputs

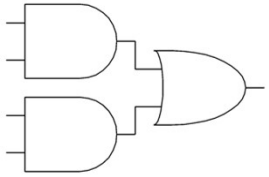




5

Rules of Combinational Composition

- Every element is combinational
- Every node is either an input or connects to *exactly one* output
- The circuit contains no cyclic paths
- **Example:**



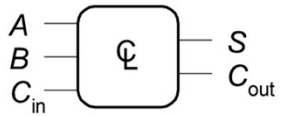


6

Boolean Equations

- Functional specification of outputs in terms of inputs
- **Example:** $S = F(A, B, C_{in})$

  $C_{out} = F(A, B, C_{in})$



$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = AB + AC_{in} + BC_{in}$$



7

Some Definitions

- **Complement**: variable with a bar over it
  $\bar{A}$, $\bar{B}$, $\bar{C}$
- **Literal**: variable or its complement
  A, $\bar{A}$, B, $\bar{B}$, C, $\bar{C}$
- **Implicant**: product of literals
  AB$\bar{C}$, $\bar{A}\bar{C}$, BC
- **Minterm**: product that includes all input variables
  AB$\bar{C}$, ABC, ABC
- **Maxterm**: sum that includes all input variables
  (A+B+C), ($\bar{A}$+B+$\bar{C}$), ($\bar{A}$+B+C)



8

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Standard Forms: Minterms & Maxterms

- In Boolean algebra, the words “product” and “sum” specify the logical operations AND and OR, respectively.
- A truth table = A Boolean function.
- It can be derived from the table by finding a logical sum of product terms for which the function output is logic 1.
- It can be derived from the table by finding a logical product of sum terms for which the function output is logic 0.



Standard Forms: Minterms & Maxterms

- A product term with all literals appearing exactly once, as complemented or uncomplemented form, is called a **minterm**.
- It represents exactly one combination of binary variable values in the truth table. It has the value 1 for that combination and 0 for all others. There are 2^n distinct minterms for n variables.



X	Y	Z	Product Term	Symbol	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
0	0	0	$\overline{X}\,\overline{Y}\,\overline{Z}$	m_0	1	0	0	0	0	0	0	0
0	0	1	$\overline{X}\,\overline{Y}Z$	m_1	0	1	0	0	0	0	0	0
0	1	0	$\overline{X}Y\overline{Z}$	m_2	0	0	1	0	0	0	0	0
0	1	1	$\overline{X}YZ$	m_3	0	0	0	1	0	0	0	0
1	0	0	$X\overline{Y}\,\overline{Z}$	m_4	0	0	0	0	1	0	0	0
1	0	1	$X\overline{Y}Z$	m_5	0	0	0	0	0	1	0	0
1	1	0	$XY\overline{Z}$	m_6	0	0	0	0	0	0	1	0
1	1	1	XYZ	m_7	0	0	0	0	0	0	0	1

MINTERMS for 3 Variable Boolean Functions

Some Properties of Minterms

- There are 2^n minterms for n Boolean variables. These minterms can be generated from the binary numbers from 0 to 2^n - 1.
- Any Boolean function can be expressed as a logical sum of minterms.b
- The complement of a function contains those minterms not included in the original function.
- A function that includes all the 2^n minterms is equal to logic 1.
- A function that is not in the sum-of-minterms form can be converted to that form by means of a truth table, since the truth table always specifies the minterms.



Standard Forms: Minterms & Maxterms

- A sum term with all literals appearing exactly once, as complemented or uncomplemented form, is called a **maxterm**.
- It represents exactly one combination of binary variable values in the truth table. It has the value 0 for that combination and 1 for all others. There are 2^n distinct maxterms for n variables.



X	Y	Z	Product Term	Symbol	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
0	0	0	$\overline{X}\,\overline{Y}\,\overline{Z}$	m_0	1	0	0	0	0	0	0	0
0	0	1	$\overline{X}\,\overline{Y}Z$	m_1	0	1	0	0	0	0	0	0
0	1	0	$\overline{X}Y\overline{Z}$	m_2	0	0	1	0	0	0	0	0
0	1	1	$\overline{X}YZ$	m_3	0	0	0	1	0	0	0	0
1	0	0	$X\overline{Y}\,\overline{Z}$	m_4	0	0	0	0	1	0	0	0
1	0	1	$X\overline{Y}Z$	m_5	0	0	0	0	0	1	0	0
1	1	0	$XY\overline{Z}$	m_6	0	0	0	0	0	0	1	0
1	1	1	XYZ	m_7	0	0	0	0	0	0	0	1

MAXTERMS for 3 Variable Boolean Functions

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Properties of Minterms & Maxterms

- A minterm is a function, not equal to 0, having the minimum number of 1s in its truth table;
- A maxterm is a function, not equal to 1, having the maximum of 1s in its truth table.
- A minterm and maxterm with the same subscript are the complements of each other; that is, $M_j = m_j'$ and $m_j = M_j'$.
- A Boolean function can be represented by forming the logical sum of all the minterms that produce a 1 from a given truth table.
- Or by forming the logical product of all the maxterms that produce a 0 from a given truth table
- Maxterms are seldom used directly when dealing with Boolean functions, since we can always replace them with the minterm list of F'.



13

Sum-of-Products (SOP) Form

- All equations can be written in SOP form
- Each row has a **minterm**
- A minterm is a product (AND) of literals
- Each minterm is TRUE for that row (and only that row)
- Form function by ORing minterms where output is 1
- Thus, a sum (OR) of products (AND terms)

A	B	Y	minterm	minterm name
0	0	0	$\overline{A}\,\overline{B}$	m_0
0	1	1	$\overline{A}\,B$	m_1
1	0	0	$A\,\overline{B}$	m_2
1	1	1	$A\,B$	m_3

$Y = F(A, B) =$



14

Sum-of-Products (SOP) Form

- All equations can be written in SOP form
- Each row has a **minterm**
- A minterm is a product (AND) of literals
- Each minterm is TRUE for that row (and only that row)
- Form function by ORing minterms where output is 1
- Thus, a sum (OR) of products (AND terms)

A	B	Y	minterm	minterm name
0	0	0	$\overline{A}\,\overline{B}$	m_0
0	1	1	$\overline{A}\,B$	m_1
1	0	0	$A\,\overline{B}$	m_2
1	1	1	$A\,B$	m_3

$Y = F(A, B) = \overline{A}B + AB = \Sigma m(1, 3)$



15

Product-of-Sums (POS) Form

- All Boolean equations can be written in POS form
- Each row has a **maxterm**
- A maxterm is a sum (OR) of literals
- Each maxterm is **FALSE** for that row (and only that row)
- Form function by ANDing maxterms where output is **0**
- Thus, a product (AND) of sums (OR terms)

A	B	Y	maxterm	maxterm name
0	0	0	$A + B$	M_0
0	1	1	$A + \overline{B}$	M_1
1	0	0	$\overline{A} + B$	M_2
1	1	1	$\overline{A} + \overline{B}$	M_3

$Y = F(A, B) =$



16

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Product-of-Sums (POS) Form

- All Boolean equations can be written in POS form
- Each row has a **maxterm**
- A maxterm is a sum (OR) of literals
- Each maxterm is **FALSE** for that row (and only that row)
- Form function by ANDing maxterms where output is **0**
- Thus, a product (AND) of sums (OR terms)

A	B	Y	maxterm	maxterm name
0	0	0	$A + B$	M_0
0	1	1	$A + \overline{B}$	M_1
1	0	0	$\overline{A} + B$	M_2
1	1	1	$\overline{A} + \overline{B}$	M_3

$$Y = F(A, B) = (A + B)(\overline{A} + B) = \Pi M(0, 2)$$



17

Boolean Equations Example

- You are going to the cafeteria for lunch
  - You won't eat lunch ($\overline{E}$)
  - If it's not open ($\overline{O}$) or
  - If they only serve corn dogs (C)
- Write a truth table for determining if you will eat lunch (E).

O	C	E
0	0	0
0	1	0
1	0	1
1	1	0



18

SOP & POS Form

SOP – sum-of-products

O	C	E	minterm
0	0	0	$\overline{O}\,\overline{C}$
0	1	0	$\overline{O}\,C$
1	0	1	$O\,\overline{C}$
1	1	0	$O\,C$

$$E = O\overline{C}$$
$$= \Sigma(2)$$

POS – product-of-sums

O	C	E	maxterm
0	0	0	$O + C$
0	1	0	$O + \overline{C}$
1	0	1	$\overline{O} + C$
1	1	0	$\overline{O} + \overline{C}$

$$E = (O + C)(O + \overline{C})(\overline{O} + \overline{C})$$
$$= \Pi(0, 1, 3)$$



19

Boolean Algebra

- Axioms and theorems to **simplify** Boolean equations
- Like regular algebra, but simpler: variables have only two values (1 or 0)
- **Duality** in axioms and theorems:
  - ANDs and ORs, 0's and 1's interchanged



20

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Boolean Algebra

- A Boolean algebra is defined as a closed algebraic system containing a set **K** that consists of **two or more elements** and the **two operators**, **AND(.)** and **OR(+)**. Useful for identifying and minimizing logic circuit functionality.

Truth Table for the Function $L = D\overline{X} + A$

D	X	A	L
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1



21

Boolean Algebra

- Boolean equations are composed of binary variables and logic operations
- Boolean functions have only one unique representation in truth table form, but many in algebraic equation forms.
- Boolean expressions dictate how the logic gates will be connected.
- They can be manipulated to obtain a simpler expression and smaller number of gates.
- Boolean functions can have one or multiple outputs.
- When two or more variables are written together it implies AND operator.

22

Boolean Axioms

Number	Axiom	Name
A1	X = 0 if X ≠ 1	Binary Field
A2	$\overline{0} = 1$	NOT
A3	0 • 0 = 0	AND/OR
A4	1 • 1 = 1	AND/OR
A5	0 • 1 = 1 • 0 = 0	AND/OR

Dual: Replace: • with +
0 with 1

23

Boolean Algebra

DUALITY PRINCIPLE: if an expression is valid in Boolean algebra, the dual of that expression is also valid. To form the dual of an expression, replace all OR operators with AND operators, all AND operators with OR operators, all 1s with 0s, and all 0s with 1s.

24

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Boolean Algebra

DUALITY PRINCIPLE: if an expression is valid in Boolean algebra, the dual of that expression is also valid. To form the dual of an expression, replace all OR operators with AND operators, all AND operators with OR operators, all 1s with 0s, and all 0s with 1s.

- **Example**: Find the dual of the Boolean expression **X + (YZ) = (X+Y)(X+Z)**



25

Boolean Algebra

DUALITY PRINCIPLE: if an expression is valid in Boolean algebra, the dual of that expression is also valid. To form the dual of an expression, replace all OR operators with AND operators, all AND operators with OR operators, all 1s with 0s, and all 0s with 1s.

- **Example**: Find the dual of the Boolean expression **X + (YZ) = (X+Y)(X+Z)**
  Solution: **X . (Y + Z) = (X . Y) + (X . Z)** Take care to keep the locations of the parenthesis if present



26

DUALITY of Minterms and Maxterms

- A minterm is a function, not equal to 0, having the minimum number of 1s in its truth table;
- A maxterm is a function, not equal to 1, having the maximum of 1s in its truth table.
- A minterm and maxterm with the same subscript are the complements of each other; that is, $M_j = m_j'$ and $m_j = M_j'$.
- A Boolean function can be represented by forming the logical sum of all the minterms that produce a 1 from a given truth table.



27

DUALITY of Minterms and Maxterms

- A minterm is a function, not equal to 0, having the minimum number of 1s in its truth table;
- A maxterm is a function, not equal to 1, having the maximum of 1s in its truth table.
- A minterm and maxterm with the same subscript are the complements of each other; that is, Mj = mj' and mj = Mj'.
- A Boolean function can be represented by forming the logical sum of all the minterms that produce a 1 from a given truth table.



28

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Boolean Axioms

Number	Axiom	Dual	Name
A1	X = 0 if X ≠ 1	X = 1 if X ≠ 0	Binary Field
A2	$\overline{0} = 1$	$\overline{1} = 0$	NOT
A3	0 • 0 = 0	1 + 1 = 1	AND/OR
A4	1 • 1 = 1	0 + 0 = 0	AND/OR
A5	0 • 1 = 1 • 0 = 0	1 + 0 = 0 + 1 = 1	AND/OR

Dual: Replace: • with +
0 with 1



29

Boolean Theorems of One Variable

Number	Theorem	Name
T1	X • 1 = X	Identity
T2	X • 0 = 0	Null Element
T3	X • X = X	Idempotency
T4	$\overline{\overline{X}} = X$	Involution
T5	X • $\overline{X}$ = 0	Complements

Dual: Replace: • with +
0 with 1



30

Boolean Theorems of One Variable

Number	Theorem	Dual	Name
T1	X • 1 = X	X + 0 = X	Identity
T2	X • 0 = 0	X + 1 = 1	Null Element
T3	X • X = X	X + X = X	Idempotency
T4	$\overline{\overline{X}} = X$		Involution
T5	X • $\overline{X}$ = 0	X + $\overline{X}$ = 1	Complements

Dual: Replace: • with +
0 with 1

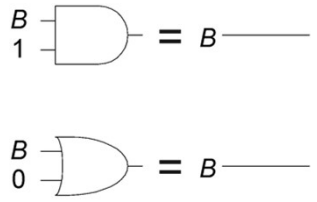


31

T1: Identity Theorem

B • 1 = B

B + 0 = B





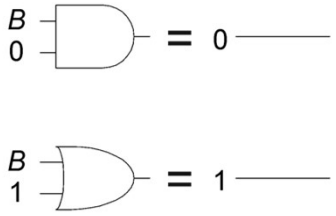
32

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

T2: Null Element Theorem

$$B \cdot 0 = 0$$

$$B + 1 = 1$$

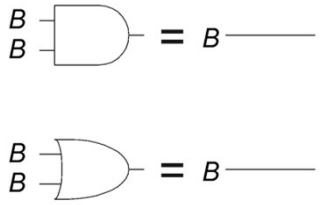


33

T3: Idempotency Theorem

$$B \cdot B = B$$

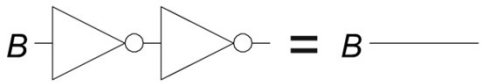
$$B + B = B$$



34

T4: Involution Theorem

$$\overline{\overline{B}} = B$$



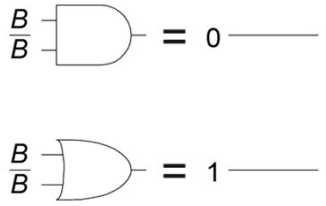
- Even multiples of complement yields the initial variable.
- Odd multiples of complement yields the complement of the variable.

35

T5: Complement Theorem

$$B \cdot \overline{B} = 0$$

$$B + \overline{B} = 1$$



36

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Boolean Theorems of Several Variables

Number	Theorem	Name
T6	X•Y = Y•X	Commutativity
T7	(X•Y) • Z = X • (Y • Z)	Associativity
T8	X • (Y + Z) = (X•Y) + (X•Z)	Distributivity
T9	X• (X+Y) = X	Covering
T10	(X•Y) + (X•$\overline{Y}$) = X	Combining
T11	(X•Y) + ($\overline{X}$•Z) + (Y•Z) = (X•Y) + ($\overline{X}$•Z)	Consensus



37

Boolean Theorems of Several Vars

#	Theorem	Dual	Name
T6	X•Y = Y•X	X+Y= Y+X	Commutativity
T7	(X•Y) • Z = X • (Y•Z)	(X + Y) + Z = X + (Y + Z)	Associativity
T8	X • (Y + Z) = (X•Y) + (X•D)	X + (Y•Z) = (X+Y) (X+Z)	Distributivity
T9	X • (X+Y) = X	X + (X•Y) = X	Covering
T10	(X•Y) + (X•$\overline{Y}$) = X	(X+Y) • (X+$\overline{Y}$) = X	Combining
T11	(X•Y) + ($\overline{X}$•Z) + (Y•Z) = (X•Y) + ($\overline{X}$•Z)	(X+Y) • ($\overline{X}$+Z) • (Y+Z) = (X+Y) • ($\overline{X}$+Z)	Consensus

Dual: Replace: • with +
0 with 1



38

Boolean Theorems of Several Vars

#	Theorem	Dual	Name
T6	X•Y = Y•X	X+Y= Y+X	Commutativity
T7	(X•Y) • Z = X • (Y•Z)	(X + Y) + Z = X + (Y + Z)	Associativity
T8	X • (Y + Z) = (X•Y) + (X•D)	X + (Y•Z) = (X+Y) (X+Z)	Distributivity
T9	X • (X+Y) = X	X + (X•Y) = X	Covering
T10	(X•Y) + (X•$\overline{Y}$) = X	(X+Y) • (X+$\overline{Y}$) = X	Combining
T11	(X•Y) + ($\overline{X}$•Z) + (Y•Z) = (X•Y) + ($\overline{X}$•Z)	(X+Y) • ($\overline{X}$+Z) • (Y+Z) = (X+Y) • ($\overline{X}$+Z)	Consensus

Axioms and theorems are useful for *simplifying* equations.



39

Boolean Algebra

PROOF OF COVERING THEOREM:

X.(X + Y) =
= X.X + X.Y =
= X + X.Y =
= X.(1 + Y) =
= X



40

Boolean Algebra

PROOF OF COMBINING THEOREM:

(X.Y) + (X.Y') =

= X.(Y + Y') =

= X.1 =

= X



41

Boolean Algebra

PROOF OF CONSENSUS THEOREM:

W = XY + X'Z + YZ

W = XY + X'Z + YZ.1

W = XY + X'Z + YZ.(X + X')

W = XY + X'Z + XYZ + X'YZ

W = XY(1 + Z) + X'Z(1 + Y)

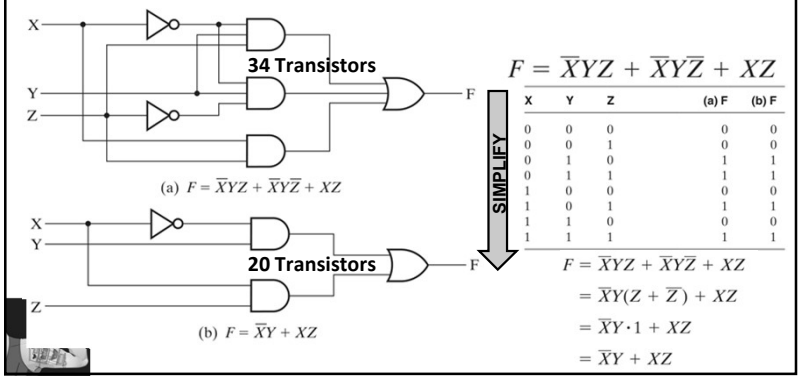
W = XY + X'Z



42

Simplifying Boolean Equations

Boolean Algebra can be used to reduce **number of gates/transistors** in a logic circuit



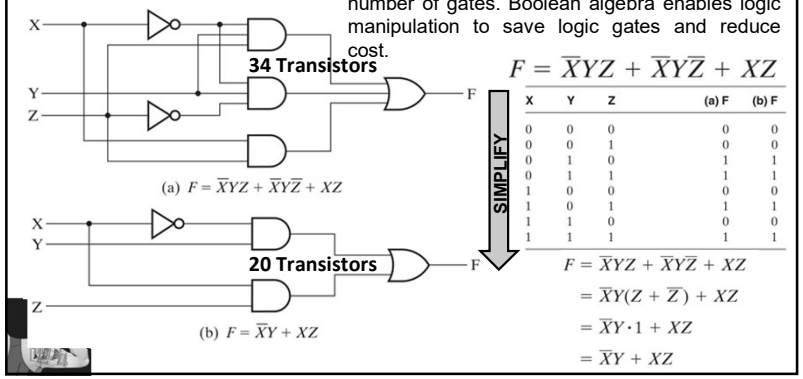


43

Simplifying Boolean Equations

Boolean Algebra can be used to reduce **number of gates/transistors** in a logic circuit

Same logic functionality is obtained using lower number of gates. Boolean algebra enables logic manipulation to save logic gates and reduce cost.



44

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Simplifying an Equation

Reducing an equation to the **fewest number of implicants**, where each implicant has the **fewest literals**

Recall:

- Implicant: product of literals
  $A\bar{B}C$, $\bar{A}C$, $\bar{B}C$
- Literal: variable or its complement
  A, $\bar{A}$, B, $\bar{B}$, C, $\bar{C}$

*Also called **minimizing** the equation*



45

Simplifying Boolean Equations

Example:

$Y = A(AB + ABC)$



46

Simplifying Boolean Equations

Example:

$Y = A(AB + ABC)$

$= A(AB(1 + C))$	T8: Distributivity
$= A(AB(1))$	T2': Null Element
$= A(AB)$	T1: Identity
$= (AA)B$	T7: Associativity
$= AB$	T3: Idempotency



47

Simplifying Boolean Equations

Example:

Y = AB'C + ABC + A'BC



48

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Simplifying Boolean Equations

Example:

Y = AB'C + ABC + A'BC

= AB'C + **ABC + ABC** + A'BC T3': Idempotency

= (AB'C+ABC) + (ABC+A'BC) T7': Associativity

= AC + BC T10: Combining



Simplifying Boolean Equations

Example:

Y = AB + BC +B'D' + AC'D'



Simplifying Boolean Equations

Example:

Y = AB + BC +B'D' + AC'D'

Method 1:

Y = AB + BC + B'D' + (ABC'D' + AB'C'D') T10: Combining

= (AB + ABC'D') + BC + (B'D' + AB'C'D') T6: Commutativity

T7: Associativity

= AB + BC + B'D' T9: Covering

Method 2:

Y = **AB** + BC + **B'D'** + AC'D' + **AD'** T11: Consensus

= AB + BC + B'D' + AD' T9: Covering

= AB + BC + B'D' T11: Consensus



Simplifying Boolean Equations

Example:

Y = (A + BC)(A + DE)



Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Simplifying Boolean Equations

Example:

Y = (A + BC)(A + DE)

Apply T8' first when possible: W+XZ = (W+X)(W+Z)

Make: X = BC, Z = DE and rewrite equation

Y	= (A+X)(A+Z)	substitution (X=BC, Z=DE)
	= A + XZ	T8': Distributivity
	= A + BCDE	substitution

or

Y	= AA + ADE + ABC + BCDE	T8: Distributivity
	= A + ADE + ABC + BCDE	T3: Idempotency
	= **A + ADE** + ABC + BCDE	
	= **A** + ABC + BCDE	T9': Covering
	= A + BCDE	T9': Covering



53

Simplifying Boolean Equations

Example:

Y = (A + BC)(A + DE)

Apply T8' first when possible: W+XZ = (W+X)(W+Z)

Make: X = BC, Z = DE and rewrite equation

Y	= (A+X)(A+Z)	substitution (X=BC, Z=DE)
	= A + XZ	T8': Distributivity
	= A + BCDE	substitution

or

Y	= AA + ADE + ABC + BCDE	T8: Distributivity
	= A + ADE + ABC + BCDE	T3: Idempotency
	= **A + ADE** + ABC + BCDE	
	= **A** + ABC + BCDE	T9': Covering
	= A + BCDE	T9': Covering

This is called ***multiplying out*** an expression to get sum-of-products (SOP) form.



54

Multiplying Out: SOP Form

An expression is in **sum-of-products (SOP)** form when all products contain literals only.

- SOP form: Y = AB + BC' + DE
- **NOT** SOP form: Y = DF + E(A'+B)
- SOP form: Z = A + BC + DE'F



55

Multiplying Out: SOP Form

Example:

Y = (A + C + D + E)(A + B)

Apply T8' first when possible: W+XZ = (W+X)(W+Z)

Make: X = (C+D+E), Z = B and rewrite equation

Y	= (A+X)(A+Z)	substitution (X=(C+D+E), Z=B)
	= A + XZ	T8': Distributivity
	= A + (C+D+E)B	substitution
	= A + BC + BD + BE	T8: Distributivity

or

Y	= AA+AB+AC+BC+AD+BD+AE+BE	T8: Distributivity
	= **A**+AB+AC+AD+AE+BC+BD+BE	T3: Idempotency
	= A + BC + BD + BE	T9': Covering



56

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Review: SOP & POS Form

- **SOP – sum-of-products**

O	C	E	minterm
0	0	0	$\overline{O}\ \overline{C}$
0	1	0	$\overline{O}\ C$
1	0	1	$O\ \overline{C}$
1	1	0	$O\ C$

$$E = O\overline{C}$$
$$= \Sigma(2)$$

- **POS – product-of-sums**

O	C	E	maxterm
0	0	0	$O + C$
0	1	0	$O + \overline{C}$
1	0	1	$\overline{O} + C$
1	1	0	$\overline{O} + \overline{C}$

$$E = (O + C)(O + \overline{C})(\overline{O} + \overline{C})$$
$$= \Pi(0, 1, 3)$$



Factoring: POS Form

An expression is in **product-of-sums (POS)** form when all sums contain literals only.

- POS form: Y = (A+B)(C+D)(E'+F)
- **NOT** POS form: Y = (D+E)(F'+GH)
- POS form: Z = A(B+C)(D+E')



Factoring: POS Form

Example 1:

Y = (A + B'CDE)

Apply T8' first when possible: W+XZ = (W+X)(W+Z)

Make: X = B'C, Z = DE and rewrite equation

Y	= (A+XZ)	substitution (X=B'C, Z=DE)
	= (A+B'C)(A+DE)	T8': Distributivity
	= (A+B')(A+C)(A+D)(A+E)	T8': Distributivity



Factoring: POS Form

Example 2:

Y = AB + C'DE + F

Apply T8' first when possible: W+XZ = (W+X)(W+Z)

Make: W = AB, X = C', Z = DE and rewrite equation

Y	= (W+XZ) + F	substitution W = AB, X = C', Z = DE
	= (W+X)(W+Z) + F	T8': Distributivity
	= (AB+C')(AB+DE)+F	substitution
	= (A+C')(B+C')(AB+D)(AB+E)+F	T8': Distributivity
	= (A+C')(B+C')(A+D)(B+D)(A+E)(B+E)+F	T8': Distributivity
	= (A+C'+F)(B+C'+F)(A+D+F)(B+D+F)(A+E+F)(B+E+F)	T8': Distributivity



Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

DeMorgan's Theorem

Number	Theorem	Name
T12	$\overline{X_0 \bullet X_1 \bullet X_2 \ldots} = \overline{X_0} + \overline{X_1} + \overline{X_2} \ldots$	DeMorgan's Theorem

The **complement** of the **product**
is the
sum of the **complements**



DeMorgan's Theorem: Dual

#	Theorem	Dual	Name
T12	$\overline{X_0 \bullet X_1 \bullet X_2 \ldots} = \overline{X_0} + \overline{X_1} + \overline{X_2} \ldots$	$\overline{X_0 + X_1 + X_2 \ldots} = \overline{X_0} \bullet \overline{X_1} \bullet \overline{X_2} \ldots$	DeMorgan's Theorem

The complement of the product
is the
sum of the complements.

Dual: The **complement** of the **sum**
is the
product of the **complements**.



DeMorgan's Theorem

The generalized form of this theorem states that the complement of an expression is obtained by interchanging AND and OR operations and complementing each variable and constant.

$$\overline{X_1 + X_2 + \ldots + X_n} = \overline{X}_1 \overline{X}_2 \ldots \overline{X}_n$$

$$\overline{X_1 X_2 \ldots X_n} = \overline{X}_1 + \overline{X}_2 + \ldots + \overline{X}_n$$

$$\overline{A + B + C + D} = \overline{A}\,\overline{B}\,\overline{C}\,\overline{D}$$

$$\overline{X \cdot Y} = \overline{X} + \overline{Y}$$

$$\overline{X + Y} = \overline{X} \cdot \overline{Y}$$

Truth Tables to Verify DeMorgan's Theorem

(a) X	Y	X + Y	$\overline{X+Y}$	(b) X	Y	$\overline{X}$	$\overline{Y}$	$\overline{X} \cdot \overline{Y}$
0	0	0	1	0	0	1	1	1
0	1	1	0	0	1	1	0	0
1	0	1	0	1	0	0	1	0
1	1	1	0	1	1	0	0	0



DeMorgan's Theorem

A simpler method for deriving the complement of a function is to take the dual of the function equation and complement each literal.

$$F_1 = \overline{X}YZ + X\overline{Y}Z \text{ and } F_2 = X(\overline{Y}\,\overline{Z} + YZ)$$

$$\overline{F}_1 = \overline{\overline{X}YZ + X\overline{Y}Z} = \overline{(\overline{X}YZ)} \cdot \overline{(X\overline{Y}Z)}$$

$$= (X + \overline{Y} + Z)(X + Y + \overline{Z})$$

$$\overline{F}_2 = \overline{X(\overline{Y}\,\overline{Z} + YZ)} = \overline{X} + \overline{(\overline{Y}\,\overline{Z} + YZ)}$$

$$= \overline{X} + \overline{\overline{Y}\,\overline{Z}} \cdot \overline{YZ}$$

$$= \overline{X} + (Y + Z)(\overline{Y} + \overline{Z})$$

We begin with

$$F_1 = \overline{X}YZ + X\overline{Y}Z = (\overline{X}YZ) + (X\overline{Y}Z)$$

The dual of F_1 is

$$(\overline{X} + Y + \overline{Z})(\overline{X} + \overline{Y} + Z)$$

Complementing each literal, we have

$$(X + \overline{Y} + Z)(X + Y + \overline{Z}) = \overline{F}_1$$

Now,

$$F_2 = X(\overline{Y}\,\overline{Z} + YZ) = X((\overline{Y}\,\overline{Z}) + (YZ))$$

The dual of F_2 is

$$X + (\overline{Y} + \overline{Z})(Y + Z)$$

Complementing each literal yields

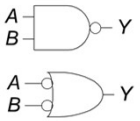
$$\overline{X} + (Y + Z)(\overline{Y} + \overline{Z}) = \overline{F}_2$$



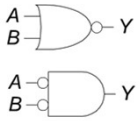
Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

DeMorgan's Theorem

- $Y = \overline{AB} = \overline{A} + \overline{B}$



- $Y = \overline{A + B} = \overline{A} \cdot \overline{B}$





65

DeMorgan's Theorem Example 1

$Y = \overline{(A+\overline{BD})\overline{C}}$



66

DeMorgan's Theorem Example 1

$Y = \overline{(A+\overline{BD})\overline{C}}$

$= \overline{(A+\overline{BD})} + \overline{\overline{C}}$

$= (\overline{A} \bullet (\overline{\overline{BD}})) + C$

$= (\overline{A} \bullet (BD)) + C$

$= \overline{A}BD + C$



67

DeMorgan's Theorem Example 2

$Y = \overline{(\overline{AC}E+\overline{D})} + B$



68

DeMorgan's Theorem Example 2

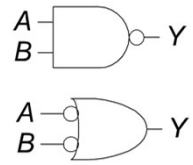
$$Y = \overline{(\overline{A}CE + \overline{D}) + B}$$
$$= \overline{(\overline{A}CE + \overline{D})} \bullet \overline{B}$$
$$= (\overline{\overline{A}CE} \bullet \overline{\overline{D}}) \bullet \overline{B}$$
$$= ((\overline{\overline{A}C} + \overline{E}) \bullet D) \bullet \overline{B}$$
$$= ((AC + \overline{E}) \bullet D) \bullet \overline{B}$$
$$= (ACD + D\overline{E}) \bullet \overline{B}$$
$$= A\overline{B}CD + \overline{B}D\overline{E}$$

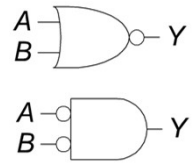


69

DeMorgan's Theorem

- $Y = \overline{AB} = \overline{A} + \overline{B}$
- $Y = \overline{A + B} = \overline{A} \cdot \overline{B}$

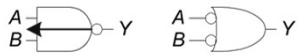




70

DeMorgan's Theorem: Bubble Pushing

- **Backward**
  - Body changes
  - Adds bubbles to inputs
- **Forward**
  - Body changes
  - Adds bubble to output

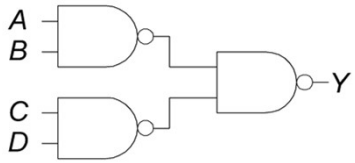




71

Bubble Pushing: Example

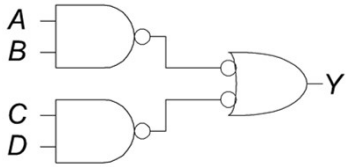
What is the Boolean expression for this circuit?



72

Bubble Pushing: Example

What is the Boolean expression for this circuit?



Y = AB + CD

Sum of Products Examples 1

We saw that a circuit behaviour can be described as Sum of Products form. Let's do a couple examples

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Sum of Products Examples 2

We saw that a circuit behaviour can be described as Sum of Products form. Let's do a couple examples

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

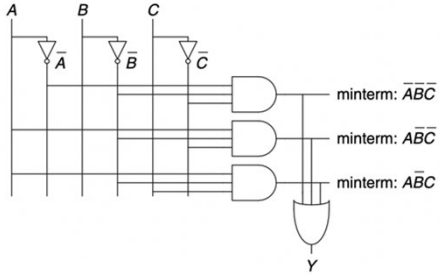
Sum of Products Examples 3

We saw that a circuit behaviour can be described as Sum of Products form. Let's do a couple examples

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

From Logic to Gates

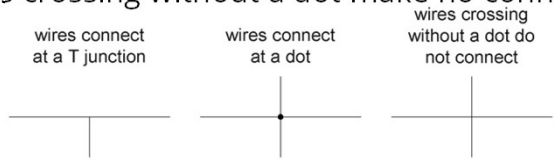
- Any Boolean equation in SoP form can be drawn as a schematic in a systematic way
- **Two-level logic :** ANDs followed by ORs
- **Example :** $Y = \bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + A\bar{B}C$



77

Circuit Schematics Rules

- Inputs on the left (or top)
- Outputs on the right (or bottom)
- Gates flow from left to right
- Straight wires are best for visualization
- Wires always connect at a T junction
- A dot where wires cross indicates a connection between the wires
- Wires crossing without a dot make no connection

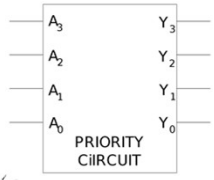


78

Multiple-Output Circuits

Assume **A CEO**, **a project manager**, **an engineer,** and **a janitor** are using the meeting room from time to time. Let's design a room reservation system:

Output is **asserted** corresponding to most significant **TRUE** input

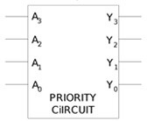


A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	**0**	**0**	**0**	**0**
0	0	0	1	**0**	**0**	**0**	**1**
0	0	1	0	**0**	**0**	**1**	**0**
0	0	1	1	**0**	**0**	**1**	**0**
0	1	0	0	**0**	**1**	**0**	**0**
0	1	0	1	**0**	**1**	**0**	**0**
0	1	1	0	**0**	**1**	**0**	**0**
0	1	1	1	**0**	**1**	**0**	**0**
1	0	0	0	**1**	**0**	**0**	**0**
1	0	0	1	**1**	**0**	**0**	**0**
1	0	1	0	**1**	**0**	**0**	**0**
1	0	1	1	**1**	**0**	**0**	**0**
1	1	0	0	**1**	**0**	**0**	**0**
1	1	0	1	**1**	**0**	**0**	**0**
1	1	1	0	**1**	**0**	**0**	**0**
1	1	1	1	**1**	**0**	**0**	**0**

79

Priority Circuit Hardware

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	**0**	**0**	**0**	**0**
0	0	0	1	**0**	**0**	**0**	**1**
0	0	1	0	**0**	**0**	**1**	**0**
0	0	1	1	**0**	**0**	**1**	**0**
0	1	0	0	**0**	**1**	**0**	**0**
0	1	0	1	**0**	**1**	**0**	**0**
0	1	1	0	**0**	**1**	**0**	**0**
0	1	1	1	**0**	**1**	**0**	**0**
1	0	0	0	**1**	**0**	**0**	**0**
1	0	0	1	**1**	**0**	**0**	**0**
1	0	1	0	**1**	**0**	**0**	**0**
1	0	1	1	**1**	**0**	**0**	**0**
1	1	0	0	**1**	**0**	**0**	**0**
1	1	0	1	**1**	**0**	**0**	**0**
1	1	1	0	**1**	**0**	**0**	**0**
1	1	1	1	**1**	**0**	**0**	**0**

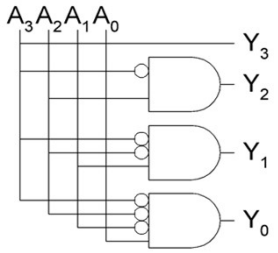


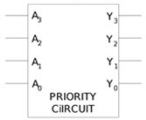
80

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Priority Circuit Hardware

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	**0**	**0**	**0**	**0**
0	0	0	1	**0**	**0**	**0**	**1**
0	0	1	0	**0**	**0**	**1**	**0**
0	0	1	1	**0**	**0**	**1**	**0**
0	1	0	0	**0**	**1**	**0**	**0**
0	1	0	1	**0**	**1**	**0**	**0**
0	1	1	0	**0**	**1**	**0**	**0**
0	1	1	1	**0**	**1**	**0**	**0**
1	0	0	0	**1**	**0**	**0**	**0**
1	0	0	1	**1**	**0**	**0**	**0**
1	0	1	0	**1**	**0**	**0**	**0**
1	0	1	1	**1**	**0**	**0**	**0**
1	1	0	0	**1**	**0**	**0**	**0**
1	1	0	1	**1**	**0**	**0**	**0**
1	1	1	0	**1**	**0**	**0**	**0**
1	1	1	1	**1**	**0**	**0**	**0**

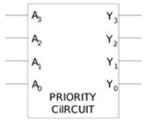




GEBZE TEKNİK ÜNİVERSİTESİ

Don't Care

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	**0**	**0**	**0**	**0**
0	0	0	1	**0**	**0**	**0**	**1**
0	0	1	0	**0**	**0**	**1**	**0**
0	0	1	1	**0**	**0**	**1**	**0**
0	1	0	0	**0**	**1**	**0**	**0**
0	1	0	1	**0**	**1**	**0**	**0**
0	1	1	0	**0**	**1**	**0**	**0**
0	1	1	1	**0**	**1**	**0**	**0**
1	0	0	0	**1**	**0**	**0**	**0**
1	0	0	1	**1**	**0**	**0**	**0**
1	0	1	0	**1**	**0**	**0**	**0**
1	0	1	1	**1**	**0**	**0**	**0**
1	1	0	0	**1**	**0**	**0**	**0**
1	1	0	1	**1**	**0**	**0**	**0**
1	1	1	0	**1**	**0**	**0**	**0**
1	1	1	1	**1**	**0**	**0**	**0**



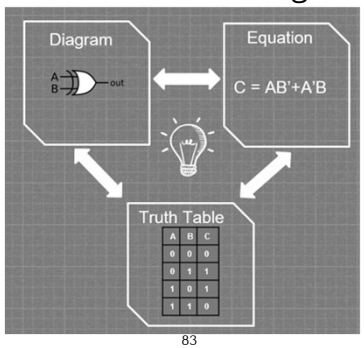
- Notice that when A_3 is asserted, the outputs **don't care** what the other inputs are.
- We use the **symbol X** to describe inputs that the output does not care about

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	**0**	**0**	**0**	**0**
0	0	0	1	**0**	**0**	**0**	**1**
0	0	1	X	**0**	**0**	**1**	**0**
0	1	X	X	**0**	**1**	**0**	**0**
1	X	X	X	**1**	**0**	**0**	**0**

GEBZE TEKNİK ÜNİVERSİTESİ

Trinity of Boolean Logic

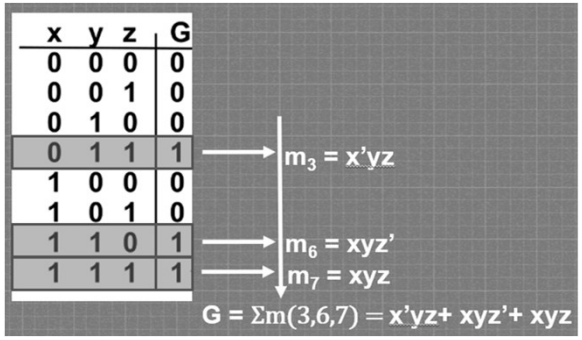
Logic diagrams, boolean equations and truth tables to represent the logic expressions. Truth tables can easily be converted to expressions. Expressions can easily be converted to two level logic diagrams.



GEBZE TEKNİK ÜNİVERSİTESİ

TRUTH TABLE TO LOGIC EXPRESSION

- Each row with an output 1 is a minterm, just add them to obtain the function as sum of products.



GEBZE TEKNİK ÜNİVERSİTESİ

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

LOGIC EXPRESSION TO TRUTH TABLE

- Use the Boolean algebra to obtain an expression of sum of products, then calculate the truth table by setting output to logic 1 for each corresponding minterm and setting the rest as logic 0.

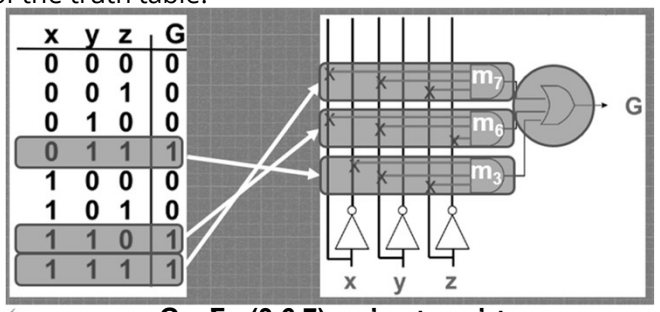
G = Σm(3,6,7) = x'yz + xyz' + xyz

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

GEBZE TEKNİK ÜNİVERSİTESİ

TRUTH TABLE TO LOGIC DIAGRAM

- Minterms will be the outputs of an AND gate, generate all minterms,
- Then use an OR gate to combine their outputs
- This would give a two level logic diagram representation of the truth table.

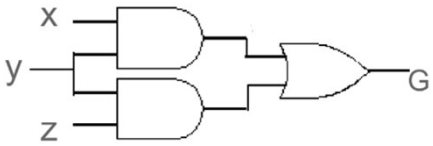


G = Σm(3,6,7) = x'yz + xyz' + xyz

GEBZE TEKNİK ÜNİVERSİTESİ

LOGIC DIAGRAM TO TRUTH TABLE

- There are 2^n-1 states for an n-input logic circuit, we apply all possible logic inputs (starting from 000..0 to 111...1) and propagate them until we get the output, then fill the table.



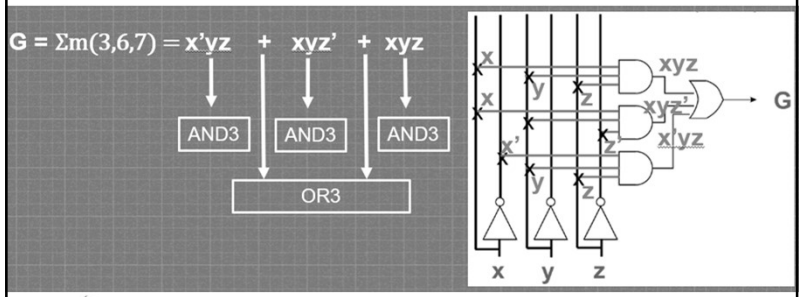
Apply xyz from 000 to 111. At each step propagate the internal signals to the final output and fill the truth table.

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

GEBZE TEKNİK ÜNİVERSİTESİ

LOGIC EXPRESSION TO LOGIC DIAGRAM

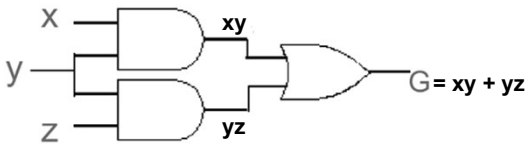
- Obtain an SOP expression, then a two level logic diagram can be drawn using AND, OR, INVERTER gates.
- In the first level product terms are generated using AND gates, and in the second level, generated product terms are combined using an OR gate.



GEBZE TEKNİK ÜNİVERSİTESİ

LOGIC DIAGRAM TO LOGIC EXPRESSION

- Apply input symbols and propagate them through the logic diagram to obtain the output function.



GEBZE TEKNİK ÜNİVERSİTESİ 89

89

END OF LECTURE

GEBZE TEKNİK ÜNİVERSİTESİ 90

90

END OF LECTURE

GEBZE TEKNİK ÜNİVERSİTESİ 91

91

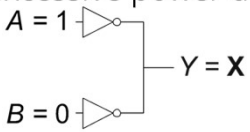
GEBZE TEKNİK ÜNİVERSİTESİ 92

92

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

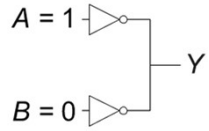
Illegal value - X

- **Contention** circuit tries to drive output to 1 **and** 0
  - Actual value somewhere in between
  - Could be 0, 1, or in forbidden zone
  - Might change with voltage, temp., time, noise
  - Often causes excessive power dissipation

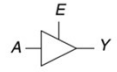


- Contention usually indicates a bug.
- X is used for both don't care and contention. Look at the context to tell them apart.

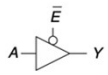
Floating - Z



- Sometimes we would like to drive the same wire with different circuits.
- We can achieve this by neither driving to HIGH nor LOW, just leave it floating.
- This is called **floating**, **high impedance** or **high Z.**
- **A tristate buffer** can be used for this purpose



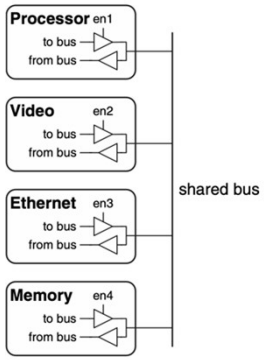
E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1



Ē	A	Y
0	0	0
0	1	1
1	0	Z
1	1	Z

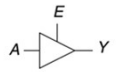
Tristate Busses

- Floating nodes are used to connect components in a bus
- A bus is used to connect components that share the same protocol using single path.
  - Many different drivers
  - Exactly one is active at once

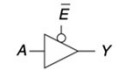


Active high, Active low

- **Active high** - when the signal or node is active when voltage level is HIGH
- **Active low** - when the signal or node is active when voltage level is LOW
- Bubble on the node denotes active low.
- In electronic circuits, many control signals are active low.



E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1

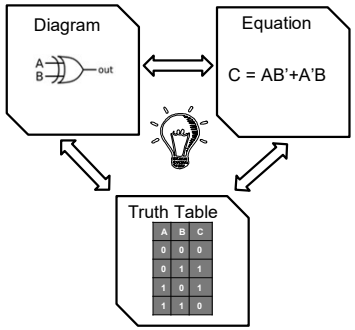


Ē	A	Y
0	0	0
0	1	1
1	0	Z
1	1	Z

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

THREE REPRESENTATIONS OF BOOLEAN LOGIC

We use logic diagrams, boolean equations and truth tables to represent the logic expressions. Truth tables can easily be converted to expressions. Expressions can easily be converted to two level logic diagrams.



97

97

TRUTH TABLE TO LOGIC EXPRESSION

Each row with an output **1** is a minterm, just add them to obtain the function as sum of products. You can also write the logic equation using product of maxterms, each row with output **0** is a maxterm, just logically AND them to obtain the product of sums representation.

LOGIC EXPRESSION TO TRUTH TABLE

Use the Boolean algebra to obtain an expression of sum of products, then calculate the truth table by setting output **1** for each corres-ponding minterm, and setting the rest as **0**.

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$m_3 = x'yz$

$m_6 = xyz'$

$m_7 = xyz$

$G = \Sigma m(3,6,7) = x'yz + xyz' + xyz$

98

98

THREE REPRESENTATIONS OF BOOLEAN LOGIC

Example: Calculate the truth table of the following Boolean expression:

F = z + x + x'(z + y')

Solution:

F = z(x + x') + x(y + y') + x'(y + y')z + x'y'(z + z')

= xz + x'z + xy + xy' + x'yz + x'y'z + x'y'z + x'y'z'

= x(y + y')z + x'(y + y')z + xy(z + z') + xy'(z + z') + x'y'z + x'y'z'

= xyz + xy'z + x'yz + x'y'z + xyz + xyz' + xy'z + xy'z' + x'y'z + x'y'z'

= ∑m(0,1,4,5,6,7)

x	y	z	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

99

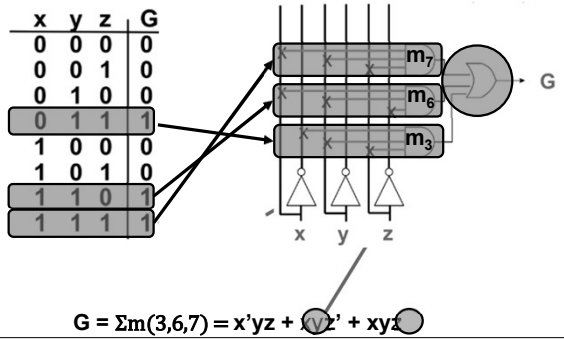
99

TRUTH TABLE TO LOGIC DIAGRAM

Each row with an output **1** is a minterm, they will be the inputs of an AND gate, generate all minterms and align them. Then use an OR gate to combine the outputs of AND gates, this would give a two level logic diagram representation of the truth table.

LOGIC DIAGRAM TO TRUTH TABLE

There are 2^N-1 states for an N input circuit, we apply all possible logic inputs (starting from 000..0 to 111…1) and propagate them until we get the output, then fill the table.



$G = \Sigma m(3,6,7) = x'yz + xyz' + xyz$

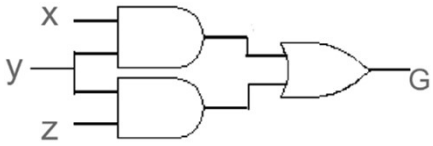
100

100

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

THREE REPRESENTATIONS OF BOOLEAN LOGIC

Example: Find the truth table for the following logic diagram:



x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Solution: Apply one by one all inputs from **000** to **111**. At each step propagate the internal signals to the final output and record it.

101

101

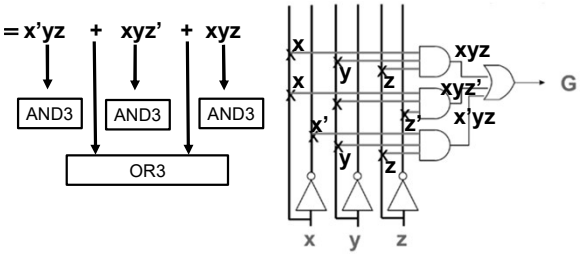
LOGIC EXPRESSION TO LOGIC DIAGRAM

Use the Boolean algebra to obtain an expression of sum of products, then a two level logic diagram can be drawn using **AND, OR, INVERTER** gates. In the first level product terms are generated using **AND** gates, and in the second level, generated product terms are combined using an **OR** gate.

LOGIC DIAGRAM TO LOGIC EXPRESSION

Apply input symbols and propagate them through the logic diagram to obtain the output function.

$G = \Sigma m(3,6,7) = x'yz + xyz' + xyz$

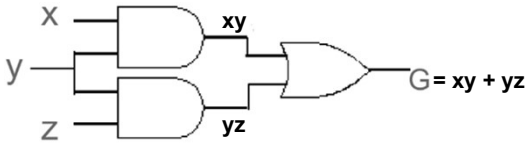


102

102

THREE REPRESENTATIONS OF BOOLEAN LOGIC

Example: Find the boolean expression for the following logic diagram:



Solution: **G = xy + yz**

103

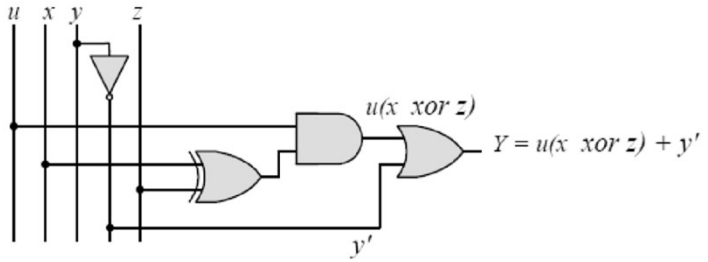
103

THREE REPRESENTATIONS OF BOOLEAN LOGIC

Example: Draw logic diagram of the following Boolean expression:

Y = u(x ⊕ z) + y'

Solution:



104

104

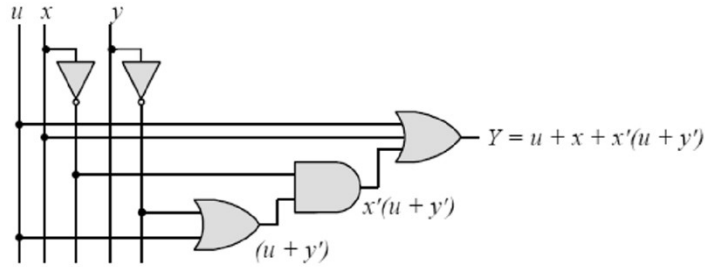
Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

THREE REPRESENTATIONS OF BOOLEAN LOGIC

Example: Draw logic diagram of the following Boolean expression:

Y = u + x + x'(u + y')

Solution:

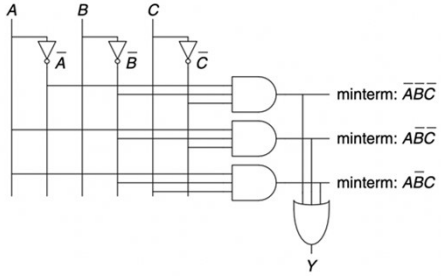


105

105

From Logic to Gates

- Any Boolean equation in SoP form can be drawn as a schematic in a systematic way
- **Two-level logic :** ANDs followed by ORs
- **Example :** Y = $\bar{A}\bar{B}\bar{C}$ + $\bar{A}\bar{B}C$ + $\bar{A}B\bar{C}$

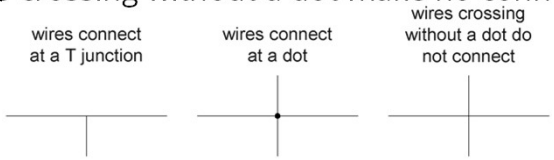


106

106

Circuit Schematics Rules

- Inputs on the left (or top)
- Outputs on the right (or bottom)
- Gates flow from left to right
- Straight wires are best for visualization
- Wires always connect at a T junction
- A dot where wires cross indicates a connection between the wires
- Wires crossing without a dot make no connection



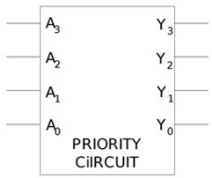
107

107

Multiple-Output Circuits

Assume **A CEO**, **a project manager**, **an engineer,** and **a janitor** are using the meeting room from time to time. Let's design a room reservation system:

Output is **asserted** corresponding to most significant **TRUE** input



A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	**0**	**0**	**0**	**0**
0	0	0	1	**0**	**0**	**0**	**1**
0	0	1	0	**0**	**0**	**1**	**0**
0	0	1	1	**0**	**0**	**1**	**0**
0	1	0	0	**0**	**1**	**0**	**0**
0	1	0	1	**0**	**1**	**0**	**0**
0	1	1	0	**0**	**1**	**0**	**0**
0	1	1	1	**0**	**1**	**0**	**0**
1	0	0	0	**1**	**0**	**0**	**0**
1	0	0	1	**1**	**0**	**0**	**0**
1	0	1	0	**1**	**0**	**0**	**0**
1	0	1	1	**1**	**0**	**0**	**0**
1	1	0	0	**1**	**0**	**0**	**0**
1	1	0	1	**1**	**0**	**0**	**0**
1	1	1	0	**1**	**0**	**0**	**0**
1	1	1	1	**1**	**0**	**0**	**0**

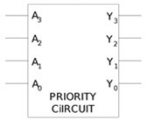
108

108

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Priority Circuit Hardware

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	**0**	**0**	**0**	**0**
0	0	0	1	**0**	**0**	**0**	**1**
0	0	1	0	**0**	**0**	**1**	**0**
0	0	1	1	**0**	**0**	**1**	**0**
0	1	0	0	**0**	**1**	**0**	**0**
0	1	0	1	**0**	**1**	**0**	**0**
0	1	1	0	**0**	**1**	**0**	**0**
0	1	1	1	**0**	**1**	**0**	**0**
1	0	0	0	**1**	**0**	**0**	**0**
1	0	0	1	**1**	**0**	**0**	**0**
1	0	1	0	**1**	**0**	**0**	**0**
1	0	1	1	**1**	**0**	**0**	**0**
1	1	0	0	**1**	**0**	**0**	**0**
1	1	0	1	**1**	**0**	**0**	**0**
1	1	1	0	**1**	**0**	**0**	**0**
1	1	1	1	**1**	**0**	**0**	**0**

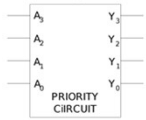


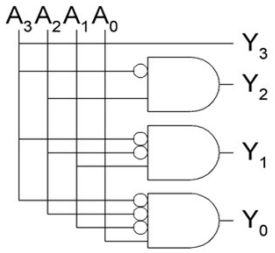
109

109

Priority Circuit Hardware

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	**0**	**0**	**0**	**0**
0	0	0	1	**0**	**0**	**0**	**1**
0	0	1	0	**0**	**0**	**1**	**0**
0	0	1	1	**0**	**0**	**1**	**0**
0	1	0	0	**0**	**1**	**0**	**0**
0	1	0	1	**0**	**1**	**0**	**0**
0	1	1	0	**0**	**1**	**0**	**0**
0	1	1	1	**0**	**1**	**0**	**0**
1	0	0	0	**1**	**0**	**0**	**0**
1	0	0	1	**1**	**0**	**0**	**0**
1	0	1	0	**1**	**0**	**0**	**0**
1	0	1	1	**1**	**0**	**0**	**0**
1	1	0	0	**1**	**0**	**0**	**0**
1	1	0	1	**1**	**0**	**0**	**0**
1	1	1	0	**1**	**0**	**0**	**0**
1	1	1	1	**1**	**0**	**0**	**0**



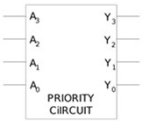


110

110

Don't Care

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	**0**	**0**	**0**	**0**
0	0	0	1	**0**	**0**	**0**	**1**
0	0	1	0	**0**	**0**	**1**	**0**
0	0	1	1	**0**	**0**	**1**	**0**
0	1	0	0	**0**	**1**	**0**	**0**
0	1	0	1	**0**	**1**	**0**	**0**
0	1	1	0	**0**	**1**	**0**	**0**
0	1	1	1	**0**	**1**	**0**	**0**
1	0	0	0	**1**	**0**	**0**	**0**
1	0	0	1	**1**	**0**	**0**	**0**
1	0	1	0	**1**	**0**	**0**	**0**
1	0	1	1	**1**	**0**	**0**	**0**
1	1	0	0	**1**	**0**	**0**	**0**
1	1	0	1	**1**	**0**	**0**	**0**
1	1	1	0	**1**	**0**	**0**	**0**
1	1	1	1	**1**	**0**	**0**	**0**



- Notice that when A_3 is asserted, the outputs **don't care** what the other inputs are.
- We use the **symbol X** to describe inputs that the output does not care about

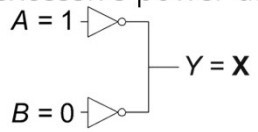
A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	**0**	**0**	**0**	**0**
0	0	0	1	**0**	**0**	**0**	**1**
0	0	1	X	**0**	**0**	**1**	**0**
0	1	X	X	**0**	**1**	**0**	**0**
1	X	X	X	**1**	**0**	**0**	**0**

111

111

Illegal value - X

- **Contention** circuit tries to drive output to 1 **and** 0
  - Actual value somewhere in between
  - Could be 0, 1, or in forbidden zone
  - Might change with voltage, temp., time, noise
  - Often causes excessive power dissipation



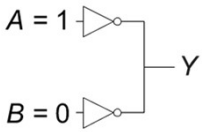
- Contention usually indicates a bug.
- X is used for both don't care and contention. Look at the context to tell them apart.

112

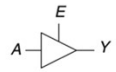
112

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

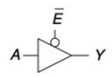
Floating - Z



- Sometimes we would like to drive the same wire with different circuits.
- We can achieve this by neither driving to HIGH nor LOW, just leave it floating.
- This is called **floating**, **high impedance** or **high Z.**
- **A tristate buffer** can be used for this purpose



E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1



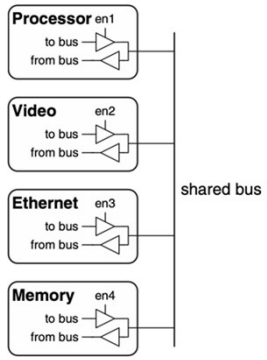
Ē	A	Y
0	0	0
0	1	1
1	0	Z
1	1	Z

113

113

Tristate Busses

- Floating nodes are used to connect components in a bus
- A bus is used to connect components that share the same protocol using single path.
  - Many different drivers
  - Exactly one is active at once

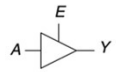


114

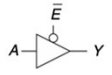
114

Active high, Active low

- **Active high** - when the signal or node is active when voltage level is HIGH
- **Active low** - when the signal or node is active when voltage level is LOW
- Bubble on the node denotes active low.
- In electronic circuits, many control signals are active low.



E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1



Ē	A	Y
0	0	0
0	1	1
1	0	Z
1	1	Z

115

115

BOOLEAN LOGIC EXPRESSION MINIMIZATION

We can use Boolean algebra postulates to reduce complexity of logic expressions while preserving the same functionality. For instance let us take a closer look at the following truth table:

A	B	C	T1	T2
0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	0	1

There are three main methods for minimization

–Algebraic minimization

–Karnaugh Map minimization

–Quine-McCluskey (tabular) minimization

T1 = $\sum m(0,1,2)$, and T2 = $\sum m(3,4,5,6,7)$,

T1 = A'B'C' + A'B'C + A'BC' = A'B'(C + C') + A'BC'
= A'(B' + BC') = A'((B + B') (B'+C')) = **A'(B' + C')**

T2 = T1' = A'BC + AB'C' + AB'C + ABC' + ABC
= BC(A + A') + AB'(C' + C) + AB(C' + C)
= BC + AB' + AB = BC + A (B' + B) = **A + BC**

116

116

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Ders Bilgisi: Haftalık Program

- Hafta 1 - Derse Giriş ve Numara Sistemleri
- Hafta 2 - Mantık Kapıları ve **Boolean Cebri**
- Hafta 3 - Birleşimli Mantık Devreleri ve Donanım Sadeleştirme
- Hafta 4 - Donanım Tanımlama Dillerine Giriş
- Hafta 5 - Eşzamanlı Mantık Tasarımı
- Hafta 6 - Aritmetik Mantık Devreleri
- Hafta 7 - Sıralı Mantık Devreleri
- Hafta 8 - Tekrar, Ara sınav
- Hafta 9 - Sonlu Otomatlar
- Hafta 10 - Hafıza ve Lojik Dizileri
- Hafta 11 - Mimari ve Mikromimari
- Hafta 12 - Tek Ritimli İşlemci
- Hafta 13 - Çok Ritimli İşlemci
- Hafta 14 - Hafıza, Önbellek ve Sanal Hafıza

BOOLEAN ALGEBRA

DUALITY

The principle of duality is an important concept. This says that if an expression is valid in Boolean algebra, the dual of that expression is also valid. To form the dual of an expression, replace all **OR** operators with **AND** operators, all **AND** operators with **OR** operators, all **1**s with **0**s, and all **0**s with **1**s.

Example: Find dual of the Boolean expression **X + (YZ) = (X+Y)(X+Z)**
Solution: X . (Y + Z) = (X . Y) + (X . Z)

Take care to keep the locations of the parenthesis if present

INVOLUTION

Since **X . X' = 0** and **X + X' = 1**, **X'** is the complement of **X** and **X** is also the complement of **X'**, as the complement of **X'** is unique it follows that **(X')' = X.**

Even multiples of complement yields the initial variable.

Odd multiples of complement yields the complement of the variable.

ABSORPTION

X + X . Y = X, also X . (X + Y) = X (Can you prove it?)